pyfieldml: a modern Python implementation of the FieldML 0.5 standard for computational biomechanics

Francis Chemorion[a,*]

[a] *BCN MedTech, Universitat Pompeu Fabra, Roc Boronat 138, 08018 Barcelona, Spain*

Abstract

FieldML is a declarative standard for representing mathematical fields over finite-element meshes, developed under the Physiome Project. Its reference C++ implementation has been largely unmaintained since 2015, leaving modern Python biomechanics pipelines without a native FieldML path. `pyfieldml` is a pure-Python reimplementation of FieldML 0.5 that exposes a full evaluator-graph DOM, an evaluation engine covering Lagrange and cubic-Hermite bases on the six standard topologies, a ten-dataset musculoskeletal model zoo, interoperability bridges to meshio, PyVista, XDMF and scikit-fem, a command-line interface with validation linter, and an in-browser JupyterLite demo. The package is `pip`-installable, Apache-2.0 licensed, and has 220+ continuously-integrated tests on Python 3.10–3.13 across Linux, macOS and Windows.

Keywords: FieldML, Physiome Project, finite element, musculoskeletal modelling, computational biomechanics, Python

Current code version

Current executable software version

1. Motivation and significance

Computational physiology and biomechanics depend on faithful, reusable descriptions of geometric and physical fields defined over finite-element meshes: bone and soft-tissue geometry, muscle-fibre orientations, bone-mineral density, cardiac conductivity tensors. The Physiome Project [1], coordinated

*Corresponding author.
Email address: `francis.chemorion@upf.edu` (Francis Chemorion)

Nr.	Code metadata description	Value
C1	Current code version	v1.2.0
C2	Permanent GitHub link to code/repository used for this code version	`https://github.com/kchemorion/pyfieldml`
C3	Legal Code License	Apache License 2.0
C4	Code versioning system used	git
C5	Software code languages, tools, and services used	Python 3.10+, NumPy, h5py, lxml, meshio, PyVista, scikit-fem
C6	Compilation requirements, operating environments & dependencies	Python 3.10+ (Linux, macOS, Windows); optional HDF5 support via h5py; optional visualisation via PyVista/VTK
C7	If available, link to developer documentation/manual	`https://github.com/kchemorion/pyfieldml#readme`
C8	Support email for questions	`francis.chemorion@upf.edu`

Table 1: Code metadata (mandatory).

Nr.	(Executable) Software metadata description	Value
S1	Current software version	v1.2.0
S2	Permanent link to executables of this version	`https://pypi.org/project/pyfieldml/`
S3	Legal Software License	Apache License 2.0
S4	Computing platforms/Operating Systems	Python wheel installable via `pip install pyfieldml`; Linux, macOS, Windows
S5	Installation requirements & dependencies	Python 3.10+; NumPy, lxml, h5py; optional extras `[meshio,viz,skfem,xdmf,all]`
S6	If available, link to user manual	`https://github.com/kchemorion/pyfieldml#readme`
S7	Support email for questions	`francis.chemorion@upf.edu`

Table 2: Software metadata (mandatory).

from the Auckland Bioengineering Institute, proposed FieldML as the standard representation for such data. The conceptual paper by Christie et al. [2] introduces FieldML as "a declarative language for building hierarchical models from reusable, self-contained components," in which a document is an explicit graph of typed *evaluators* (parameter, external, aggregate, piecewise,

reference) over discrete *ensemble* and continuous domains. This evaluator-graph view is powerful: it captures not just a mesh and a field but the entire mathematical pipeline that produces field values from parametric coordinates, including basis functions, scale factors, and connectivity.

The reference C++ implementation of FieldML, FieldML-API [3], was developed alongside the 0.5 specification and is tightly coupled to the OpenCMISS solver infrastructure [4]. It depends on libxml2 and HDF5, must be built from source, exposes no Python bindings, and has seen little maintenance since approximately 2015. Meanwhile, the computational-physiology and biomechanics communities have moved decisively toward Python: meshio [5] for format interoperability, PyVista [6] for visualisation, scikit-fem [7] for lightweight FEM assembly, and the OpenSim [8, 9] family of tools for musculoskeletal (MSK) dynamics are now standard building blocks. No native Python path into FieldML currently exists. As a consequence, the substantial corpus of Physiome-Project models encoded in FieldML 0.3, 0.4 and 0.5 is effectively invisible to modern Python pipelines, and new investigators must choose between ad-hoc formats or a steep C++ build.

`pyfieldml` closes this gap. It is a pure-Python, independent reimplementation of FieldML 0.5 that can be installed with `pip install pyfieldml` in a single step, with no compilation required. It reads and writes FieldML 0.3, 0.4 and 0.5 (up-converting older versions transparently), evaluates fields at arbitrary parametric coordinates using a plugin-based basis-function registry, and round-trips documents through meshio, PyVista, XDMF3 and scikit-fem. The concepts of the 2009 paper [2] map directly onto the package's Python class hierarchy: `ParameterEvaluator`, `ExternalEvaluator`, `AggregateEvaluator`, `PiecewiseEvaluator` and `ReferenceEvaluator` are first-class objects composed into a `Document`. Related open-source mesh infrastructure— libMesh [10] and the FEniCS, deal.II and MOOSE ecosystems—provides excellent general-purpose FEM, but none speaks FieldML natively, and none encodes the evaluator-graph abstraction that makes FieldML valuable as an *interchange* format.

The experimental setting for which `pyfieldml` is designed is the day-to-day biomechanics modelling workflow: load a geometry from a bundled or Physiome-Project dataset, inspect its evaluator graph, evaluate fields over elements, pass the mesh to a solver (such as scikit-fem or a FEniCS front-end), visualise the result, and export to VTK/XDMF for further analysis. The package has already been used internally at Universitat Pompeu Fabra to ingest BodyParts3D [11] skeletal geometries into musculoskeletal pipelines, and it powers the first in-browser FieldML viewer through a JupyterLite [12] site that requires no local installation.

2. Software description

2.1. Software architecture

`pyfieldml` is organised as a stack of layered subpackages (Fig. 1). The lowest layer, `dom`, is a thin wrapper over `lxml` [13] that parses and writes FieldML XML against the bundled XSD schemas for versions 0.3, 0.4 and 0.5, and implements an up-converter that normalises older documents to the 0.5 shape on read. The `model` layer turns that XML tree into a semantic object graph whose nodes are typed evaluators, following the nomenclature of Christie et al. [2]. A `Document` exposes mapping-style accessors—`doc.evaluators`, `doc.region.continuous`, `doc.region.meshes`—that make the evaluator graph directly navigable from Python.

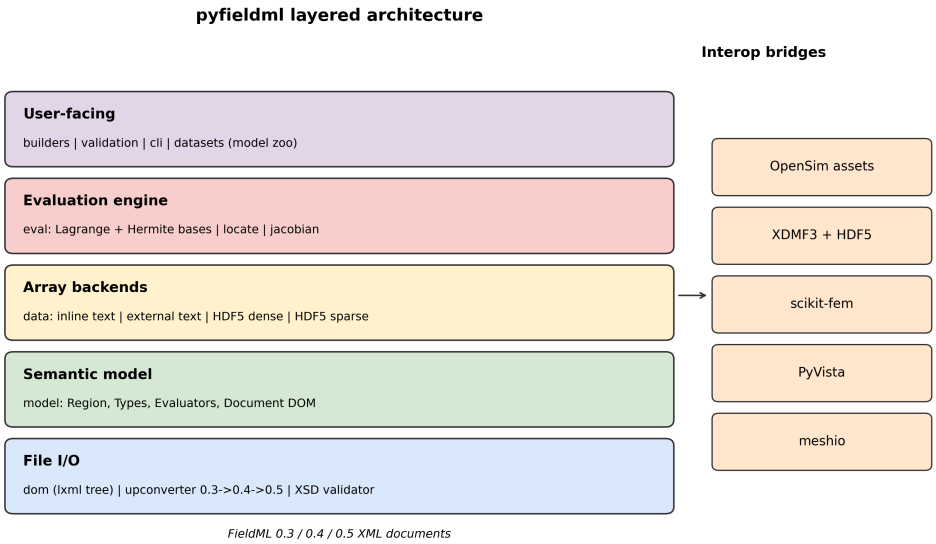

Figure 1: Layered architecture of `pyfieldml`. The XML DOM layer handles FieldML 0.3/0.4/0.5 read/write and version up-conversion. The semantic model layer exposes a typed evaluator graph. The array-backend layer provides four independent storage paths (inline, external text, HDF5 dense, HDF5 sparse). The evaluation engine implements Lagrange and Hermite bases with spatial locate and Jacobian. The user-facing layer bundles builders, a validation linter, a CLI, and the ten-dataset model zoo. Interop bridges connect to the Python scientific stack.

Array data (parameter values, connectivity tables) is handled by the `data` layer, which provides four interchangeable backends—inline text, external text file, dense HDF5 [14, 15], and sparse HDF5 dictionary-of-keys—all exposed through a common `ndarray`-like interface with lazy loading and round-trip guarantees for `float32`, `float64`, `int32` and `int64`. The `eval` layer im-

plements the evaluation engine: a plugin-based basis registry (12 Lagrange bases covering line/triangle/quadrilateral/tetrahedron/hexahedron/wedge at linear and quadratic order, and three cubic-Hermite bases on line/quad/hex with per-node scale factors), a spatial locate routine that inverts world → xi by Newton iteration, and a Jacobian calculator. `Field.evaluate(element, xi)`, `Field.sample(points)` and `Field.jacobian(element, xi)` are vectorised over NumPy [16] arrays.

The user-facing layer contains `builders` (high-level constructors for common mesh/field patterns), a `validation` linter that flags orphan evaluators and inverted elements, a `cli` (`pyfieldml inspect | validate | convert | plot | lint | diff | bench`), and the `datasets` model zoo. Interop bridges are implemented as optional extras: `meshio` (two-way plugin), `PyVista` (`doc.plot()`), `scikit-fem` (returning a `(Mesh, Basis)` tuple ready for PDE assembly), `XDMF3`, and an OpenSim-adjacent asset exporter.

2.2. Software functionalities

The key functionalities of `pyfieldml` are: (i) read/write of FieldML 0.3/0.4/0.5 with transparent up-conversion, validated against the bundled XSD; (ii) an evaluator-graph DOM whose typed objects mirror the specification; (iii) an evaluation engine spanning 15 basis functions over six topologies with vectorised `evaluate`, `sample` and `jacobian`; (iv) a ten-dataset bundled model zoo (synthetic `unit_cube`, `femur`, `rectus_femoris`; Stanford Bunny [17]; six real BodyParts3D [11] skeletal segments: `femur_bodyparts3d`, `tibia_left`, `hip_bone_left`, `vertebra_l3`, `scapula`, `skull`) shipped inside the wheel; (v) interop bridges to meshio, PyVista, XDMF3 and scikit-fem; (vi) a CLI and semantic linter; (vii) a SHA256-stamped reproducibility benchmark and a conformance workflow that cross-checks I/O against the C++ reference; (viii) an in-browser JupyterLite demo.

2.3. Sample code snippet

A complete load → evaluate → export round-trip fits in ten lines:

```python
import pyfieldml as fml

doc = fml.datasets.load("femur_bodyparts3d") # BodyParts3D mesh
coords = doc.field("coordinates") # evaluator graph
xyz = coords.evaluate(element=1, xi=(0.33, 0.33))
grid = doc.to_pyvista() # PyVista mesh
doc.to_meshio().write("femur.vtu") # VTU export
fml.validation.lint(doc) # catch orphans
```

No FieldML-specific boilerplate is required; the document, its evaluator graph and its array data behave like first-class Python objects.

3. Illustrative examples

The three illustrative examples in this section are drawn from the eight CI-executed tutorial notebooks that ship with `pyfieldml`.

3.1. Example 1: The bundled model zoo

`pyfieldml` ships with ten curated datasets inside its wheel (Fig. 2). They range from a 8-node `unit_cube` sanity check, through synthetic but anatomically-plausible `femur` and `rectus_femoris` meshes, to the public-domain Stanford Bunny and six real human skeletal segments harvested from BodyParts3D [11], a CC-BY-SA 2.1 JP anatomical database curated by the Database Center for Life Science (DBCLS) in Japan. A researcher can enumerate, load and visualise the entire zoo in three lines:

```
from pyfieldml import datasets
for name in datasets.list():
    datasets.load(name).plot()
```

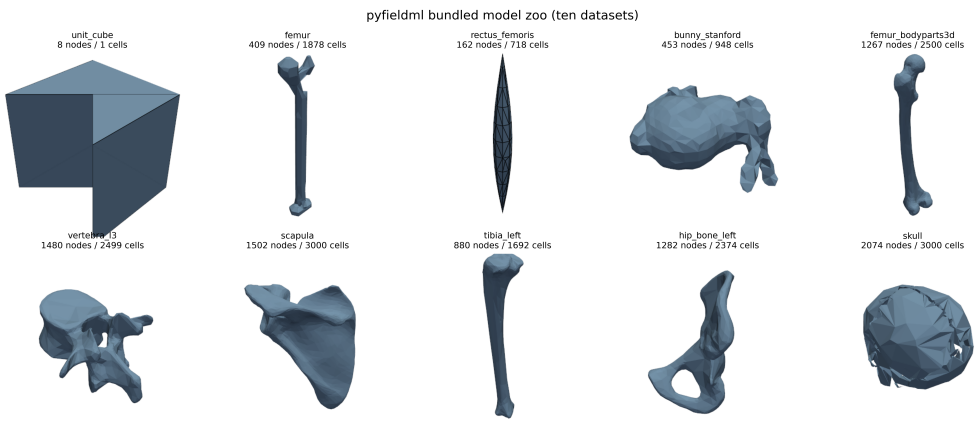

Figure 2: The ten bundled datasets shipped inside the `pyfieldml` wheel. Top row (synthetic and public-domain): `unit_cube`, `femur`, `rectus_femoris`, Stanford Bunny [17], `femur_bodyparts3d`. Bottom row (BodyParts3D [11], CC-BY-SA 2.1 JP): `vertebra_l3`, `scapula`, `tibia_left`, `hip_bone_left`, `skull`. Node and cell counts shown below each panel.

The zoo is designed to lower the barrier to reproducible biomechanics experiments: every published pyfieldml workflow can point to a bundled as-

set rather than to an external download. Each dataset carries a machine-readable `DatasetInfo` record (SPDX license, citation, origin URL) enabling downstream tooling to surface provenance correctly.

Fig. 3 illustrates the *evaluator graph* of one dataset, `rectus_femoris`. Nodes are typed evaluators and edges are named dependencies, exactly in the sense of Christie et al. [2]. This graph view is produced by a single-call interactive widget (`doc.explore()`) inside a Jupyter notebook, or rendered with NetworkX for publication, and makes FieldML's structure visible to users who may never have opened a FieldML XML file.

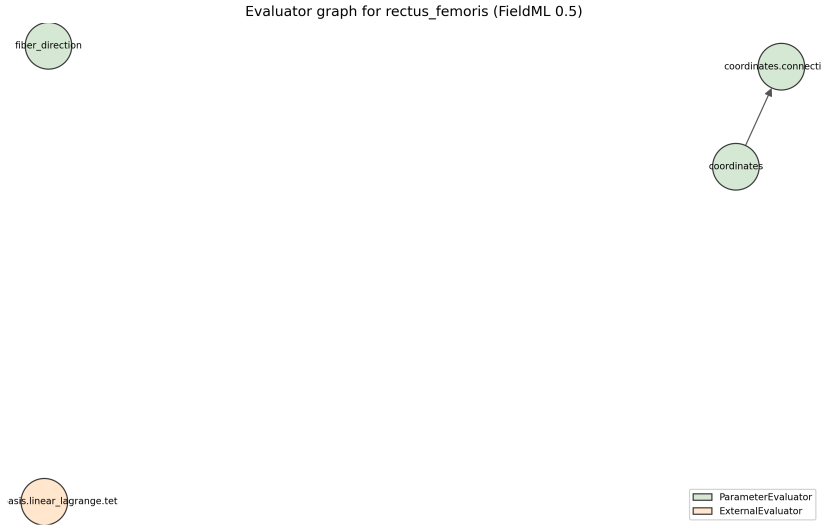

Figure 3: Evaluator graph of the bundled `rectus_femoris` dataset. Nodes are coloured by evaluator subtype (parameter / external / aggregate / reference / piecewise / argument / constant) following the FieldML 0.5 taxonomy. Edges are named dependencies: the `coordinates` parameter depends on its `coordinates.connectivity` ensemble, which in turn binds into the element topology via an `ExternalEvaluator` basis.

3.2. Example 2: FEM solve via scikit-fem

A FieldML document can be handed directly to scikit-fem [7] for PDE assembly through the `pyfieldml.interop.scikit_fem` bridge, which returns a `(Mesh, Basis)` pair. The Poisson problem $-\Delta u = 1$ on a unit cube, for instance, reduces to a standard scikit-fem boilerplate once the mesh has been obtained from FieldML (Fig. 4). The pattern generalises to any FieldML mesh whose basis is supported by scikit-fem, including the triangular-surface BodyParts3D bones. Crucially, neither the user nor the solver is aware that the mesh began life as a FieldML 0.5 evaluator graph—the bridge is transparent.

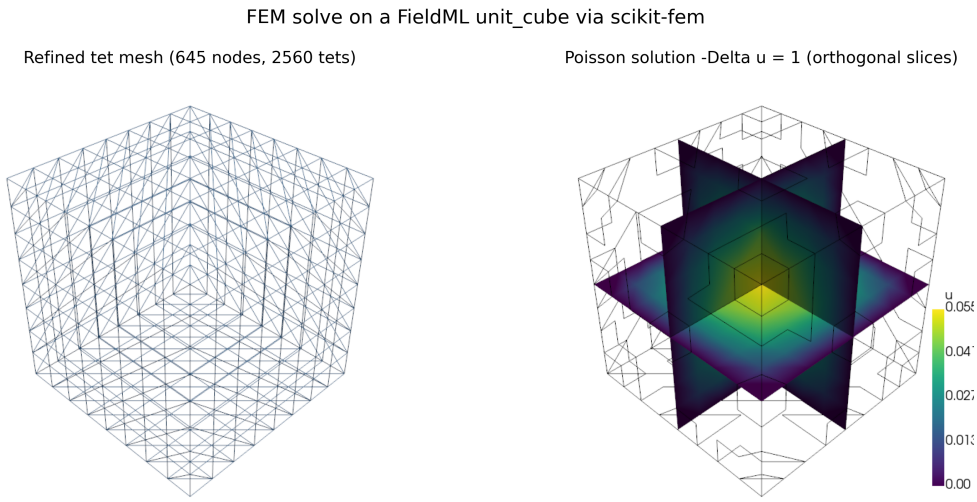

Figure 4: FEM workflow starting from a FieldML mesh. Left: a refined tetrahedral mesh derived from the bundled `unit_cube` via the `pyfieldml` → `scikit-fem` bridge. Right: Poisson solution $-\Delta u = 1$ with zero Dirichlet boundary, rendered on three orthogonal slices.

3.3. Example 3: Clinical musculoskeletal assembly

The most important consumer of `pyfieldml` is the computational biomechanics community. Fig. 5 shows a real-world use case: four BodyParts3D [11] skeletal segments—left hip bone, left femur, left tibia, and lumbar vertebra L3—loaded in a single Python session, composed in a shared coordinate frame, and rendered together. The entire assembly is constructed from bundled datasets, with no external downloads and no C++ build, in roughly ten lines of Python. For MSK researchers this replaces what would otherwise be a multi-tool pipeline involving external STL exporters, manual scaling, and format conversions.

Beyond assembly, the Hermite basis of the evaluation engine enables smooth-deformation studies. Fig. 6 shows a two-node cubic-Hermite line element, the simplest setting in which the C1-continuous Hermite basis is visible, loaded as a tip-force cantilever. pyfieldml samples the deflection at 41 xi points; the resulting curve is C1-continuous by construction, matching the theoretical Euler-Bernoulli profile. The same Hermite machinery is what enables smooth heart-wall geometries in Physiome-Project cardiac models [18].

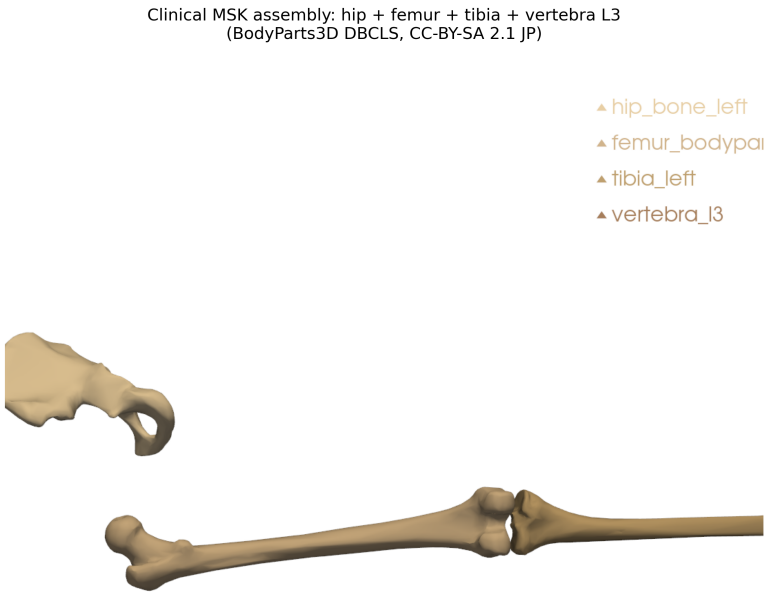

Figure 5: Clinical musculoskeletal assembly rendered from four bundled BodyParts3D [11] datasets (left hip, left femur, left tibia, lumbar vertebra L3). The scene is assembled in roughly ten lines of Python on top of `pyfieldml.datasets`. BodyParts3D data is © The Database Center for Life Science (DBCLS), licensed under CC-BY-SA 2.1 JP.

4. Impact

`pyfieldml` has three immediate impacts on the computational-physiology and biomechanics communities.

Re-opening the Physiome Project corpus to modern Python. FieldML is the *lingua franca* of the Physiome Project [1, 2] and is the native format of a substantial corpus of published models distributed through the Auckland Bioengineering Institute and the OpenCMISS ecosystem [4]. That corpus has been locked behind a C++ build for a decade. By providing a pure-Python reader with transparent 0.3 → 0.4 → 0.5 up-conversion, `pyfieldml` lets any Python researcher load a FieldML document from the Physiome Model Repository and feed it directly into a modern

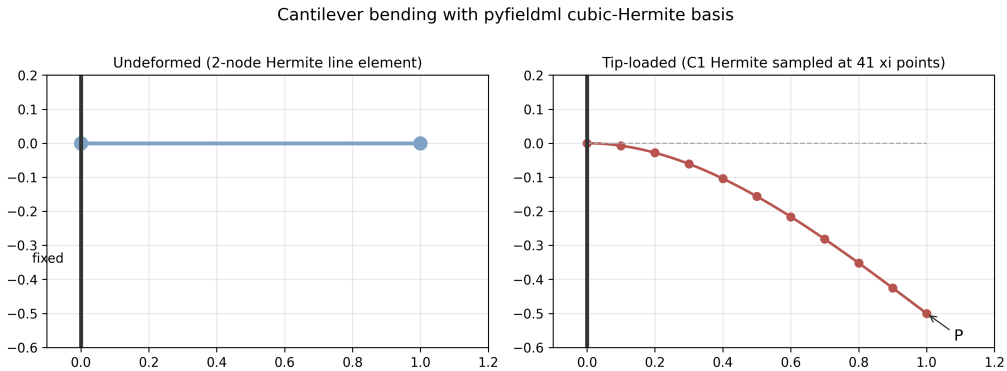

Figure 6: Cantilever bending with the `pyfieldml` cubic-Hermite basis. Left: undeformed two-node Hermite line element. Right: tip-loaded deflection, sampled at 41 parametric (ξ) points via `Field.evaluate`. The profile is C1-continuous by basis construction.

pipeline (PyVista [6], scikit-fem [7], FEniCS, OpenSim [8, 9]). The same `fml.datasets.load(name).to_meshio()` call that drives this paper's figures suffices.

Enabling reproducible musculoskeletal research workflows. The ten-dataset bundled model zoo is deliberately oriented toward the BodyParts3D [11] skeletal subset most relevant to lower-limb biomechanics. The zoo lets an MSK researcher depend on a single `pip install pyfieldml` for geometry assets and pair them directly with OpenSim [9] muscle-model plugins through the package's OpenSim-adjacent asset exporter (geometry `.vtu`, muscle attachments CSV, fibre-line polylines). At UPF we have already used this path to prototype fibre-field studies on the bundled `rectus_femoris` muscle without needing to distribute mesh assets to collaborators. The new research questions this enables—parameter studies over entire cohorts of BodyParts3D-derived meshes, reproducible benchmarks for MSK pipelines—have been impractical to pursue with the C++ FieldML-API.

First in-browser FieldML viewer. The shipped JupyterLite [12] site makes `pyfieldml` the first FieldML toolchain that can be inspected, evaluated and visualised entirely in a web browser with no local install. This lowers the barrier for teaching (a FieldML tutorial can be run as a URL) and for reviewing: a reviewer of a biomechanics paper can open the authors' JupyterLite sandbox, load the FieldML asset, and re-run the evaluation interactively. For FieldML 0.5, no such capability existed before.

The intended user communities are: the Auckland Bioengineering Institute and the broader Physiome / OpenCMISS community; OpenSim [9] and 3D Slicer [19] practitioners who now consume FieldML assets via meshio;

the MSK biomechanics group at Universitat Pompeu Fabra; computational-cardiology groups working with Nash–Hunter-style cardiac mechanics [18]; and the broader scientific-Python ecosystem that relies on PyVista [6], meshio [5] and scikit-fem [7]. The package is distributed on PyPI with OIDC-backed trusted publishing, and carries a CITATION.cff, Apache 2.0 license, and a continuous-integration matrix covering Python 3.10–3.13 across Linux, macOS and Windows. It has 220+ automated tests and a SHA256-stamped reproducibility benchmark that verifies deterministic output across platforms.

5. Conclusions

`pyfieldml` is a pure-Python reimplementation of the FieldML 0.5 standard that closes the decade-long gap between the conceptual foundation of FieldML [2] and the modern scientific-Python ecosystem. It provides a complete evaluator-graph DOM with 0.3/0.4/0.5 up-conversion, an evaluation engine spanning 15 basis functions on six topologies, a curated ten-dataset musculoskeletal model zoo, interop bridges to meshio, PyVista, XDMF and scikit-fem, a CLI with semantic linter, and the first in-browser FieldML viewer.

The immediate roadmap includes parametric statistical shape models [20] over the BodyParts3D-derived zoo, a full two-way cross-validation workflow against the C++ reference implementation, and native integration with OpenSim's musculoskeletal dynamics pipeline. All code is Apache-2.0 licensed and contributions are welcome at `https://github.com/kchemorion/pyfieldml`.

Acknowledgements

The author thanks the Physiome Project community and the original FieldML authors (G. Richard Christie, Caton Little, Alan Wu, Richard Christie, Andrew Miller, Poul Nielsen, Peter Hunter) for the conceptual foundation that `pyfieldml` builds on, and the Database Center for Life Science (DBCLS) Japan for maintaining BodyParts3D under a free license.

References

[1] P. J. Hunter, T. K. Borg, Integration from proteins to organs: the Physiome Project, Nature Reviews Molecular Cell Biology 4 (3) (2003) 237–243. `doi:10.1038/nrm1054`.

[2] G. R. Christie, P. M. F. Nielsen, S. A. Blackett, C. P. Bradley, P. J. Hunter, FieldML: concepts and implementation, Philosophical Transactions of the Royal Society A: Mathematical, Physical and Engineering Sciences 367 (1895) (2009) 1869–1884. doi:10.1098/rsta.2009.0025.

[3] C. Little, A. Wu, R. Christie, A. Miller, FieldML-API: C++ reference implementation, https://github.com/kchemorion/FieldML-API, auckland Uniservices Ltd (2015).

[4] C. Bradley, A. Bowery, R. Britten, V. Budelmann, O. Camara, R. Christie, A. Cookson, A. F. Frangi, T. B. Gamage, T. Heidlauf, S. Krittian, D. Ladd, C. Little, K. Mithraratne, M. Nash, D. Nickerson, P. Nielsen, O. Röhrle, S. Safaei, R. Sebastian, M. Steghoefer, T. Wu, T. Yu, H. Zhang, P. Hunter, OpenCMISS: A multi-physics & multi-scale computational infrastructure for the VPH/Physiome project, Progress in Biophysics and Molecular Biology 107 (1) (2011) 32–47. doi:10.1016/j.pbiomolbio.2011.06.015.

[5] N. Schlömer, meshio: I/O for various mesh formats, https://github.com/nschloe/meshio (2024). doi:10.5281/zenodo.1173115.

[6] C. B. Sullivan, A. A. Kaszynski, PyVista: 3D plotting and mesh analysis through a streamlined interface for the Visualization Toolkit (VTK), Journal of Open Source Software 4 (37) (2019) 1450. doi:10.21105/joss.01450.

[7] T. Gustafsson, G. D. McBain, scikit-fem: A Python package for finite element assembly, Journal of Open Source Software 5 (52) (2020) 2369. doi:10.21105/joss.02369.

[8] S. L. Delp, F. C. Anderson, A. S. Arnold, P. Loan, A. Habib, C. T. John, E. Guendelman, D. G. Thelen, OpenSim: Open-source software to create and analyze dynamic simulations of movement, IEEE Transactions on Biomedical Engineering 54 (11) (2007) 1940–1950. doi:10.1109/TBME.2007.901024.

[9] A. Seth, J. L. Hicks, T. K. Uchida, A. Habib, C. L. Dembia, J. J. Dunne, C. F. Ong, M. S. DeMers, A. Rajagopal, M. Millard, S. R. Hamner, E. M. Arnold, J. R. Yong, S. K. Lakshmikanth, M. A. Sherman, J. P. Ku, S. L. Delp, OpenSim: Simulating musculoskeletal dynamics and neuromuscular control to study human and animal movement, PLoS Computational Biology 14 (7) (2018) e1006223. doi:10.1371/journal.pcbi.1006223.

[10] B. S. Kirk, J. W. Peterson, R. H. Stogner, G. F. Carey, libMesh: a C++ library for parallel adaptive mesh refinement/coarsening simulations, Engineering with Computers 22 (3–4) (2006) 237–254. doi: 10.1007/s00366-006-0049-3.

[11] N. Mitsuhashi, K. Fujieda, T. Tamura, S. Kawamoto, T. Takagi, K. Okubo, BodyParts3D: 3D structure database for anatomical concepts, Nucleic Acids Research 37 (suppl_1) (2009) D782–D785. doi: 10.1093/nar/gkn613.

[12] JupyterLite Contributors, JupyterLite: Jupyter in the browser, https://jupyterlite.readthedocs.io/ (2024).

[13] S. Behnel, M. Faassen, I. Bicking, lxml: XML and HTML with Python, https://lxml.de/ (2024).

[14] The HDF Group, Hierarchical Data Format, version 5, https://www.hdfgroup.org/solutions/hdf5/ (2024).

[15] A. Collette, HDF5 for Python, https://www.h5py.org/ (2024).

[16] C. R. Harris, K. J. Millman, S. J. van der Walt, R. Gommers, P. Virtanen, D. Cournapeau, E. Wieser, J. Taylor, S. Berg, N. J. Smith, R. Kern, M. Picus, S. Hoyer, M. H. van Kerkwijk, M. Brett, A. Haldane, J. F. del Río, M. Wiebe, P. Peterson, P. Gérard-Marchant, K. Sheppard, T. Reddy, W. Weckesser, H. Abbasi, C. Gohlke, T. E. Oliphant, Array programming with NumPy, Nature 585 (7825) (2020) 357–362. doi:10.1038/s41586-020-2649-2.

[17] G. Turk, M. Levoy, Zippered polygon meshes from range images, in: Proceedings of the 21st annual conference on Computer graphics and interactive techniques (SIGGRAPH '94), 1994, pp. 311–318, dataset from Stanford 3D Scanning Repository, https://graphics.stanford.edu/data/3Dscanrep/. doi:10.1145/192161.192241.

[18] M. P. Nash, P. J. Hunter, Computational mechanics of the heart: From tissue structure to ventricular function, Kluwer Academic Publishers, Dordrecht, 2000, journal of Elasticity 61:113–141, 2000. doi:10.1023/A:1011084330767.

[19] A. Fedorov, R. Beichel, J. Kalpathy-Cramer, J. Finet, J.-C. Fillion-Robin, S. Pujol, C. Bauer, D. Jennings, F. Fennessy, M. Sonka, J. Buatti, S. Aylward, J. V. Miller, S. Pieper, R. Kikinis, 3D Slicer as an

image computing platform for the Quantitative Imaging Network, Magnetic Resonance Imaging 30 (9) (2012) 1323–1341. doi:10.1016/j.mri.2012.05.001.

[20] T. F. Cootes, C. J. Taylor, D. H. Cooper, J. Graham, Active shape models - their training and application, Computer Vision and Image Understanding 61 (1) (1995) 38–59. doi:10.1006/cviu.1995.1004.